Journal of Control and Decision

ISSN: 2330-7706 (Print) 2330-7714 (Online) Journal homepage: www.tandfonline.com/journals/tjcd20

MPC with adaptive-bias RBFNN and Q-reinforcement learning for fault-tolerant control of nonlinear processes

Jianguo Dong, Tianyu Yu, Shiyu Yang, Ruixian Su & Shijian Dong

JOURNAL OF CONTROL AND DECISION
https://doi.org/10.1080/23307706.2025.2556337

Taylor & Francis
Taylor & Francis Group

MPC with adaptive-bias RBFNN and Q-reinforcement learning for fault-tolerant control of nonlinear processes

Jianguo Dong[a], Tianyu Yu[b], Shiyu Yang[c], Ruixian Su[b] and Shijian Dong [d]

[a]School of Automation, Southeast University, Nanjing, People's Republic of China; [b]School of Information and Control Engineering, China University of Mining and Technology, Xuzhou, People's Republic of China; [c]School of Mechanical and Electrical Engineering, China University of Mining and Technology, Xuzhou, People's Republic of China; [d]School of Metallurgy, Northeastern University, Shenyang, People's Republic of China

ABSTRACT

A nonlinear model predictive control (NMPC) with a feedback fault compensator, using an adaptive-bias radial basis function neural network (RBFNN), is proposed to achieve high precision control of nonlinear processes with dynamic modelling deviations or fault disturbances. The stability control is realised by NMPC with prediction optimisation and feedback correction. An RBFNN is constructed to compensate for the lumped deviations caused by modelling errors, fault disturbances and random noise. A particle swarm optimisation algorithm with Q-reinforcement learning (QPSO) is established to obtain the optimal control parameters, and its fitness function is designed by cumulative absolute tracking error (CATE). The closed-loop stability of the proposed algorithm is analysed by constructing a Lyapunov function. The superiority of the proposed control algorithm is verified through comparative testing. The average coupling error (MCE) of the control algorithm proposed is 0.14, which has been reduced by three times compared to other algorithms.

ARTICLE HISTORY
Received 9 March 2025
Accepted 30 August 2025

KEYWORDS
Nonlinear; model predictive control; RBF neural network; Q-reinforcement learning; stability analysis; fault-tolerant control

1. Introduction

Due to modelling deviations and uncertain disturbances in industrial processes, it is impossible to establish an accurate nominal model suitable for the model predictive control (MPC) method (Bangi & Kwon, 2023; Bamimore, 2024; L. Yang et al., 2020). Industrial production processes may encounter unknown fault disturbances (Amin & Hasan, 2019; Yan et al., 2024; H. Zhang et al., 2025). These fault signals are difficult to measure directly and adversely affect controller performance (H. Zhang et al., 2024). The fault-tolerant control technology based on the intermediate state observer needs to strictly distinguish the types of faults (J.X. Zhang & Yang, 2020). An accurate state space model can facilitate observer estimation and fault compensation (Guo & Zhang, 2024). It is also necessary to establish another control strategy to reject modelling deviation and disturbance, which increases the complexity of such fault-tolerant control algorithms (W. Wu et al., 2024). Many undetermined parameters involved in the observer are difficult to set effectively (Fang, Fan, et al., 2025; Fang, Ruan, et al., 2023). In contrast, using intelligent networks to estimate and compensate for faults or unmodelled dynamics can be expected to obtain a better engineering control effect (Shabbir et al., 2023). The lumped deviations composed of the modelling deviation, fault disturbance and noise should be compensated by the intelligent network. By constructing a loss evaluation function that can reflect the control performance, an intelligent optimisation algorithm should be established to optimise the undetermined parameters.

Based on the mechanism knowledge of the nonlinear processes, the nominal model can be obtained through reasonable assumptions and simplification (Z. Wu et al., 2020). Compared with model-free control methods, model-based control technology has unique stable tracking performance and disturbance rejection ability (Al-Kaf et al., 2023). Compared to the PID algorithm, NMPC can simultaneously consider multiple variables. PID requires the use of a variable decoupling technique to control the multi-variable system. NMPC predicts the behaviour of the system over a period of time in the future. By considering input changes, system constraints and objective functions, MPC effectively optimises the future control actions. In addition, MPC with reasonable constraint processing ability can ensure the control feasibility (De Souza et al., 2010). The MPC can minimise the deviation between the predicted output trajectory and the reference trajectory in the finite time domain to ensure the control stability (Hill et al., 2021). The MPC is a local optimal control strategy suitable for nonlinear multivariable processes (Lin et al., 2020). A nonlinear robust

CONTACT Shijian Dong dsjggy@126.com School of Metallurgy, Northeastern University, Shenyang, People's Republic of China

MPC framework has been presented for handling general disturbances by using an online constructed tube to tighten the nominal constraints (Köhler et al., 2021). A distributed economic MPC method has been presented for multi-stage belt conveyor systems (C. Yang et al., 2023). It is necessary to reasonably compensate for the lumped errors composed of model mismatch, fault and disturbance.

The precise control of nonlinear processes with uncertain faults has attracted the attention of scientists (Liu et al., 2024; Wang & Yang, 2016; J.Z. Yang et al., 2022). By combining a disturbance observer and fuzzy logic system, a composite control method has been proposed for a manned submersible vehicle (Fang, Fan, et al., 2025; Fang, Ruan, et al., 2023). An off-policy reinforcement learning-based model-free minimax fault-tolerant control has been proposed to control industrial processes with disturbance and actuator failures (X. Li et al., 2022). An enhanced model reference adaptive control scheme has been designed for nonlinear processes with actuator faults (Arıcı & Kara, 2020). Since the feedforward radial basis function neural network (RBFNN) can approximate any nonlinear function with arbitrary precision within a compact set (Papadimitrakis & Alexandridis, 2022). A nonlinear predictive controller with RBFNN has been designed for controlling an electrostatic micro electro mechanical system (Rezvani & Keymasi-Khalaji, 2024). However, the standard RBFNN cannot handle dynamic model bias (Sun et al., 2022). Although increasing the number of hidden layer nodes can improve the approximate ability, it increases a large computational burden (Csekő et al., 2015). Therefore, the control performance of the MPC controller can be improved by modifying the RBFNN to compensate for the lumped error.

It is difficult to adjust the parameters in the MPC algorithm manually (Ding & Li, 2021). The parameter optimisation of the controller belongs to the problem of solving non-convex functions with constraints (Krog & Jäschke, 2024). Conventional analytic algorithms struggle to obtain satisfactory parameters (H. Zhang et al., 2020). Heuristic intelligent algorithms can be used to optimise the tuning (Júnior et al., 2014). In contrast, the particle swarm optimisation (PSO) algorithm has the advantages of simple structure, strong global search ability to high high-dimensional space (Gan et al., 2018; Diwan & Deshpande, 2023). A 2-degrees of freedom (DOF) planar robot has been controlled by a Fuzzy Logic controller tuned with a PSO algorithm (Bingül & Karahan, 2011). But the standard PSO algorithm cannot simultaneously satisfy the optimisation requirements of a complex controller (Tian & Shi, 2018). By adopting Q-learning to evaluate their accumulated performance, a QSO algorithm has been proposed (Hsieh & Su, 2016). Although QSO improves the global convergence performance of the PSO algorithm to some extent, it is easy to fall into a local extreme value due to the use of cumulative and partial performance. An improved PSO algorithm has been presented to design an ultra-wideband (UWB) antenna (Y.L. Li et al., 2013), which does not fundamentally solve the essential problems of PSO. Since the PSO algorithm with fixed parameters cannot dynamically adjust the parameters, this limitation restricts its global search ability (Tijjani et al., 2024). Inappropriate parameters may cause the PSO algorithm to converge to a local global optimal solution. Various improved PSO algorithms have been proposed, including PSO with asynchronous learning factors, hybrid-based PSO, second-order oscillatory PSO and simulated annealing-based PSO. However, these improved algorithms have not fundamentally solved the performance deficiency of the basic PSO algorithm. This article uses the Q-Learning strategy to improve the PSO algorithm. By synthesising particle state information and control error information, an improved PSO strategy can be designed that is suitable for the MPC method.

The principal contributions of the study mainly include that (1) the nominal model of nonlinear process is established by simplification and approximate expansion techniques; (2) a bias term is firstly added to the RBF hidden layer to avoid the problem of approximation failure when the input of the neural network deviates from its approximation domain; the improved adaptive-bias RBF network is designed to compensate lumped uncertainties; (3) the PSO algorithm is innovatively improved by introducing Q-reinforcement learning strategy; (4) the fitness function based on CATE is designed to ensure that the optimal parameters can achieve good tracking performance; and (5) the closed-loop stability of the proposed algorithm is also proved by constructing the Lyapunov function. The control performance of the proposed algorithm is verified by experimental testing.

The subsequent parts of this article include the following six sections. Section 2 derives the nominal model of the nonlinear system and presents the standard MPC formulation. Section 3 summarises the adaptive-bias RBFNN compensator and the convergence of its learning rule is rigorously proved. The QPSO algorithm is constructed by integrating Q-reinforcement learning into PSO in Section 4. The closed-loop stability proof is given in Section 5. Section 6 conducts comparative experiments on a two-link nonlinear manipulator. Finally, some key conclusions and future research directions are summarised in Section 7.

2. MPC of the nonlinear process

The standard MPC uses the predictive model to predict and optimise the output of the future time at each control time, which may not be directly applied to nonlinear processes. Therefore, approximate linearisation or

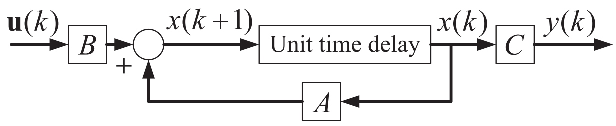

Figure 1. Block diagram of Taylor expansion model.

identification techniques can be employed to obtain the nominal reference model of the nonlinear process. A nominal dynamic linearisation model of the nonlinear processes can be established by performing a process model near the operating point. The first-order Taylor expansion model of the nonlinear processes is shown in Figure 1 and can usually be expressed as follows:

$$\begin{cases} \mathbf{x}(k+1) = A\mathbf{x}(k) + B\mathbf{u}(k) \\ \mathbf{y}(k) = C\mathbf{x}(k), \quad k = 0, 1, 2, \ldots \end{cases} \tag{1}$$

where A and B are the state matrix and the input matrix, respectively, which are usually dynamic and time-varying; $\mathbf{x}(k)$ is the state of the system; and C is the output matrix.

The nominal model is used to design the controller. By recursively applying the system state equation, the state prediction equation under the control sequence can be obtained as follows:

$$\begin{cases} \mathbf{x}(k+1|k) = A\mathbf{x}(k) + B\mathbf{u}(k|k) \\ \mathbf{x}(k+2|k) = A^2\mathbf{x}(k) + AB\mathbf{u}(k|k) + B\mathbf{u}(k+1|k) \\ \qquad \vdots \\ \mathbf{x}(k+n_c|k) = A^{n_c}\mathbf{x}(k) + \sum_{i=1}^{n_c} A^{n_c-i}B\mathbf{u}(k+i-1|k) \\ \qquad \vdots \\ \mathbf{x}(k+n_p|k) = A^{p}\mathbf{x}(k) + \sum_{i=1}^{n_c} A^{n_p-i}B\mathbf{u}(k+i-1|k) \end{cases} \tag{2}$$

where n_p is the prediction time domain; n_c is the control time domain.

In practical applications, the selection of the prediction and control time domain of the MPC algorithm should consider the dynamic characteristics of the system, the accuracy of the model and the computing resources. The prediction time domain refers to the time period used to predict future behaviour, while the control time domain refers to the time period used to implement control actions. Generally, the prediction time domain is longer than the control time domain. In the predictive time domain, the system dynamics are predicted forward at the current moment. Too short a prediction time domain may lead to insufficient dynamic prediction, which cannot adequately reflect the future behaviour of the system. Too long a prediction time domain will be affected by model mismatch and disturbance, resulting in inaccurate prediction. A reasonable length of control time domain can effectively reduce the computational burden and the number of variables to be optimised, thereby reducing the computational complexity. It is very arbitrary to set the values of the control and predict time domain solely based on experience. It is difficult to ensure the computational efficiency while also considering the control effect. Therefore, this article uses an intelligent PSO algorithm to optimise the control parameters by setting a reasonable objective function.

The output prediction equation can be expressed as follows:

$$\begin{cases} \mathbf{y}(k+1|k) = A\mathbf{x}(k) + B\mathbf{u}(k|k) \\ \mathbf{y}(k+2|k) \\ \quad = CA^2\mathbf{x}(k) + CAB\mathbf{u}(k|k) + CB\mathbf{u}(k+1|k) \\ \qquad \vdots \\ \mathbf{y}(k+n_c|k) \\ \quad = CA^{n_c}\mathbf{x}(k) + \sum_{i=1}^{n_c} CA^{n_c-1}B\mathbf{u}(k+i-1|k) \\ \qquad \vdots \\ \mathbf{y}(k+n_p|k) \\ \quad = CA^{n_p}\mathbf{x}(k) + \sum_{i=1}^{n_c} CA^{n_p-1}B\mathbf{u}(k+i-1|k) \end{cases} \tag{3}$$

where $x(k+i|k)$ and $y(k+i|k)$ are the state prediction value and output prediction value at time $k+i$, respectively, using the information at time k.

Equation (3) can be described in the following matrix form.

$$\begin{bmatrix} \mathbf{y}(k+1|k) \\ \mathbf{y}(k+2|k) \\ \vdots \\ \mathbf{y}(k+n_c|k) \\ \vdots \\ \mathbf{y}(k+n_p|k) \end{bmatrix}_{n_p\times 1} = \begin{bmatrix} CA \\ CA^2 \\ \vdots \\ CA^{n_c} \\ \vdots \\ CA^{n_p} \end{bmatrix}_{n_p\times 1} \mathbf{x}(k) + \begin{bmatrix} CB & \mathbf{O} & \mathbf{O} & \mathbf{O} \\ CAB & CB & \mathbf{O} & \mathbf{O} \\ \vdots & \vdots & \cdots & \vdots \\ CA^{n_c-1}B & CA^{n_c-2}B & \cdots & CB \\ \vdots & \vdots & \cdots & \vdots \\ CA^{n_p-1}B & CA^{n_p-2}B & \cdots & CA^{n_p-n_c}B \end{bmatrix}_{n_p\times n_c} \times \begin{bmatrix} \mathbf{u}(k|k) \\ \mathbf{u}(k+1|k) \\ \vdots \\ \mathbf{u}(k+n_c-1|k) \end{bmatrix}_{n_c\times 1} \tag{4}$$

By defining the matrix, Equation (4) can be simplified as follows:

$$\mathbf{Y}(k) = \Phi\mathbf{x}(k) + \Pi\mathbf{U}(k) \tag{5}$$

where $\mathbf{Y}(k) = [\mathbf{y}(k+1|k), \ldots, \mathbf{y}(k+n_c|k), \ldots, \mathbf{y}(k+n_p|k)]^T$, $\mathbf{U}(k) = [\mathbf{u}(k|k), \ldots, \mathbf{u}(k+n_c|k)]^T$, $\Phi =$

$[CA, \ldots, CA^{n_c}, \ldots, CA^{n_p}]^T$ and

$$\Pi = \begin{bmatrix} CB & \mathbf{O} & \mathbf{O} & \mathbf{O} \\ CAB & CB & \mathbf{O} & \mathbf{O} \\ \vdots & \vdots & \cdots & \vdots \\ CA^{n_c-1}B & CA^{n_c-2}B & \cdots & CB \\ \vdots & \vdots & \cdots & \vdots \\ CA^{n_p-1}B & CA^{n_p-2}B & \cdots & CA^{n_p-n_c}B \end{bmatrix}_{n_p \times n_c}$$

are matrices; the coefficient matrices Φ and Π are also named as prediction matrix.

The prediction matrix is recursively obtained from the state space equation of the system. For the determined prediction time domain n_p and control time domain n_c, the prediction matrix values depend on the approximate linear system parameters at the current time. As the approximate linear system is updated, the parameter values of the prediction matrix change accordingly each time. Consider the following criterion function constructed by the tracking error and input signal.

$$J(k) = \sum_{i=k+1}^{k+n_n} \theta_i ||y_d(i) - y(i|k)||^2 + \sum_{i=k+1}^{k+m} \xi_i ||u(i)||^2 \tag{6}$$

where θ_i and ξ_i are tracking error weight and input weight at time i; $y_d(i)$ is the reference output signal.

The criterion function can be described by the following equation.

$$\begin{aligned} J(k) &= [\mathbf{Y}(k) - \mathbf{Y}_d(k)]^T \Theta [\mathbf{Y}(k) - \mathbf{Y}_d(k)] \\ &\quad + \mathbf{U}^T(k) \Xi \mathbf{U}(k) \end{aligned} \tag{7}$$

where $\mathbf{Y}(k)$ is the output matrix; $\mathbf{U}(k)$ is the system control input matrix; $\mathbf{Y}_d(k) = [y_d(k), \ldots, y_d(k)]^T_{1\times n_p}$ is the designed reference output signal matrix; $\Theta = \mathrm{diag}\{\theta_1, \theta_2, \ldots, \theta_{n_p}\}$ is the output error weight matrix; $\Xi = \mathrm{diag}\{\xi_1, \xi_2, \ldots, \xi_{n_c}\}$ is the input weight matrix.

The optimal control input signal can be obtained by minimising the criterion function to achieve feedback correction. Note that a large Θ value can improve the speed of the controller to track the reference trajectory. The value of Ξ can limit the control input size. A large Ξ value will reduce the tracking rate of the reference signal. Large n_p and n_c will reduce the accuracy of prediction and control, while too small values will make the control effect worse.

By polynomial expansion of Equation (7), we get

$$\begin{aligned} J(k) &= [\Phi\mathbf{x}(k) + \Pi\mathbf{U}(k) - \mathbf{Y}_d(k)]^T \Theta [\Phi\mathbf{x}(k) \\ &\quad + \Pi\mathbf{U}(k) - \mathbf{Y}_d(k)] + U^T(k)\Xi U(k) \\ &= \mathbf{x}^T(k)\Phi^T\Theta\Phi\mathbf{x}(k) + \mathbf{x}^T(k)\Phi^T\Theta\Pi\mathbf{U}(k) \\ &\quad - \mathbf{x}^T(k)\Phi^T\Theta\mathbf{Y}_d(k) + \mathbf{U}^T(k)\Pi^T\Theta\Phi x(k) \\ &\quad + \mathbf{U}^T(k)\Pi^T\Theta\Pi\mathbf{U}(k) - \mathbf{U}^T(k)\Pi^T\Theta\mathbf{Y}_d(k) \\ &\quad - \mathbf{Y}_d{}^T(k)\Theta\Phi\mathbf{x}(k) - \mathbf{Y}_d{}^T(k)\Theta\Pi\mathbf{U}(k) \\ &\quad + \mathbf{Y}_d{}^T(k)\Theta\mathbf{Y}_d(k) + \mathbf{U}^T(k)\Xi\mathbf{U}(k) \\ &= 2(\mathbf{x}^T(k)\Phi^T - \mathbf{Y}_d{}^T(k))\Theta\Pi\mathbf{U}(k) \\ &\quad + \mathbf{U}^T(k)(\Pi^T\Theta\Pi + R)\mathbf{U}(k) + \Lambda \\ &= 2F\mathbf{U}(k) + \mathbf{U}^T(k)H\mathbf{U}(k) + \Lambda \\ &= 2F\mathbf{U}(k) + \mathbf{U}^T(k)H\mathbf{U}(k) + \Lambda \end{aligned} \tag{8}$$

where Λ consists of all the terms in the equation that are independent of $U(k)$. Λ can be safely ignored during the optimisation process and solving the input signal; The $F = (x^T(k)\Phi^T - \mathbf{Y}_d{}^T(k))\Theta\Pi$ is defined as the prediction error weighting term; The $H = \Pi^T\Theta\Pi + \Xi$ is defined as the quadratic form representing the input weights.

For solving the optimal problem $\min_{\mathbf{U}(k)} J(\mathbf{x}(k), \mathbf{U}(k), n_c, n_p)$ to obtain optimal control input $\mathbf{U}^*(k)$, it can be converted into the following quadratic programming problem.

$$\min_{\mathbf{U}(k)} 2F\mathbf{U}(k) + \mathbf{U}^T(k)H\mathbf{U}(k) \tag{9}$$

Since the prediction equation is obtained through recursive calculation, the output error will increase with the number of prediction steps. Therefore, the first value of the optimal control input is generally taken as the control input at the next moment. The optimal control input of the process under the nominal model can be expressed as follows:

$$\mathbf{u}^*_{mpc}(k) = [\mathbf{I}, \ldots, \mathrm{O}] * \mathbf{U}(k) \tag{10}$$

By substituting the control rate into the control equation and solving it iteratively, the MPC input can be obtained.

3. Compensator based on adaptive-bias RBFNN

In the actual industrial production process, actuators, sensors or component units may encounter fault disturbances. The fault signal can cause a deviation in the optimal control input and reduce the precision and stability of the tracking control. By estimating and compensating for the fault signal into the input signal of the MPC algorithm, the optimal control law can be obtained as follows:

$$\mathbf{u}(k) = \mathbf{u}^*_{mpc}(k) - \hat{f}^* \tag{11}$$

where $\hat{f}^*$ is the estimated value of the fault signal f.

In addition, the inevitable uncertainty disturbance and modelling deviation caused by local linearisation should also be estimated and compensated for in the control law. The lumped deviation consists of modelling deviation, faults and random disturbances, which need to be estimated and compensated centrally. In this article, the adaptive RBFNN will be utilised to estimate and compensate for lumped deviation. In this case, the $\hat{f}^*$ is the estimation of lumped deviation. The RBFNN consists of the input layer, hidden layer and output layer. The activation function of the hidden layer neuron $S_i(Z)$ is composed of the radial basis function, and its form is

$$S_i(Z) = \exp\left[-\frac{(Z-\mu_i)^T(Z-\mu_i)}{\sigma^2}\right], \quad i = 1, 2, \ldots, m \tag{12}$$

where $\mu_i = [\mu_{i1}, \mu_{i2}, \ldots, \mu_{iq}]$ is the central position vector of the Gaussian basis function; σ is the width of the Gaussian basis function and $Z = [Z_1, Z_2, \ldots, Z_q] \in \Omega_z$ is the input vector.

Local bias means that the biased item is added to each neuron, while global bias means that the biased item is added to the end of the hidden layer. Therefore, the hidden layer of the adaptive-bias RBFNN can be expressed as follows:

$$\Phi(Z) = [S_1(Z) + b_l, \ldots, S_m(Z) + b_l, b_g] \tag{13}$$

where b_l is local bias item;$b_g$is global bias item.

The optimal fault signal estimation value can be expressed as follows:

$$f^* = {W^*}^T S(Z) + \varepsilon(Z) \tag{14}$$

where $W^* \in R^{m\times n}$ is the weight ideal value of the network output layer; $\varepsilon(Z)$ is the optimal estimation error. When the number of hidden layer nodes is large enough, $\varepsilon(Z)$ can approach any small value. In this case $|\varepsilon(Z)| \le \bar{\varepsilon}(Z)$, $\bar{\varepsilon}(Z)$ is an arbitrarily small normal numbers.

The ideal network output layer weight can be expressed as follows:

$$W^* := \arg\min\{|f^* - \hat{W}\Phi(Z)|\} \tag{15}$$

where $\hat{W}$ is the estimate of the ideal weight.

The weight calculation learning algorithm based on the gradient descent method is given as follows:

$$\hat{W}' = \Gamma(\Phi(Z)E - \delta\hat{W}) \tag{16}$$

where $\Gamma = \mathrm{diag}\{\gamma_1, \gamma_2, \ldots, \gamma_m\}$ is the learning rate; $E = f^* - \hat{W}^T\Phi(Z)$; δ is the correction coefficient, and its expression is given as follows:

$$\delta = \begin{cases} 0, & \text{if } ||\hat{W}|| < \bar{W} \\ \bar{\delta}, & \text{if } ||\hat{W}|| \ge \bar{W} \end{cases} \tag{17}$$

Note that δ is used to improve the robustness of the adaptive RBF controller by adding pure damping to the weights during the learning process, which may reduce the approximation performance. These shortcomings can be remedied by utilising discontinuous switching δ-correction. When the weight is $||\hat{W}|| < \bar{W}$, the δ switches to 0.

Theorem 1: *For the compensation objective function f, the learning algorithm* (16) *ensures that the error of the weight estimate $W_e = W^* - \hat{W}$ is bounded.*

To make Theorem 1 easier to understand, the following variables are declared in advance. $S_i(Z)$ is activation function of the hidden layer neuron; $\mu_i = [\mu_{i1}, \mu_{i2}, \ldots, \mu_{iq}]$ is the central position vector of the Gaussian basis function; σ is the width of the Gaussian basis function; $Z = [Z_1, Z_2, \ldots, Z_q] \in \Omega_z$ is the input vector; $\Phi(Z) = [S_1(Z) + b_l, \ldots, S_m(Z) + b_l, b_g]$ is the hidden layer of the adaptive-bias RBFNN; b_l is local bias item; b_g is global bias item; f^* is optimal fault signal estimation value; $W^* \in R^{m\times n}$ is the weight ideal value of the network output layer; $\varepsilon(Z)$ is the optimal estimation error; $\hat{W}$ is the estimate of the ideal weight; $\Gamma = \mathrm{diag}\{\gamma_1, \gamma_2, \ldots, \gamma_m\}$ is the learning rate; $E = f^* - \hat{W}^T\Phi(Z)$; δ is the correction coefficient; $W_e = W^* - \hat{W}$ is error of the weight estimate; $c_1 = \bar{\delta}\,\Gamma$, $c_2 = \frac{1}{2}\bar{\varepsilon}^2 + \frac{1}{2}\bar{\delta}\bar{W}^2$ and $\chi = 2(V(0) + \frac{c_2}{c_1})$.

Proof: Consider the following Lyapunov function.

$$V = \frac{1}{2}W_e^T\Gamma^{-1}W_e \tag{18}$$

From Equation (16), we have

$$\Gamma^{-}\hat{W} = (\Phi(Z)E - \delta\hat{W}) \tag{19}$$

Based on Equation (19), taking the derivative of Equation (18), we get

$$V' = -W_e^T(\Phi(Z)E - \delta\hat{W}) \tag{20}$$

where

$$\begin{aligned} E &= F(Z) - \hat{W}^T\Phi(Z) \\ &= {W^*}^T\Phi(Z) + \varepsilon - \hat{W}^T\Phi(Z) \\ &= W_e^T\Phi(Z) \end{aligned} \tag{21}$$

By substituting Equation (21) into Equation (20), we get

$$V' = -W_e^T\Phi(Z)\Phi^T(Z)W_e - W_e^T\Phi(Z)\varepsilon - \delta W_e^T\hat{W} \tag{22}$$

Due to

$$-W_e^T\Phi(Z)\varepsilon \le \frac{1}{2}(W_e^T\Phi(Z))^2 + \frac{1}{2}\bar{\varepsilon}^2 \tag{23}$$

Equation (22) can be rewritten as

$$V' = \frac{1}{2}(W_e^T\Phi(Z))^2 + \frac{1}{2}\bar{\varepsilon}^2 - \delta W_e^T\hat{W} \tag{24}$$

From Equation (17), for $||\hat{W}|| < \bar{W}$ and $\delta = 0$, we get

$$\delta W_e^T \hat{W} = 0 \le \frac{1}{2}\bar{\delta}(\bar{W}^2 - ||W_e||^2) \tag{25}$$

When $||\hat{W}|| \ge \bar{W}$ and $\delta = \bar{\delta}$, we have

$$\begin{aligned} \delta W_e^T \hat{W} &= \bar{\delta} W_e^T (W^* - W_e) \\ &= -\bar{\delta} W_e^T W_e + \bar{\delta} W_e^T W^* \\ &\le -\frac{\bar{\delta}}{2} W_e^T W_e + \frac{\bar{\delta}}{2}||W^*||^2 \\ &\le -\frac{\bar{\delta}}{2}||W_e||^2 + \frac{\bar{\delta}}{2}\bar{W}^2 \end{aligned} \tag{26}$$

Therefore, we get

$$\delta W_e^T \hat{W} \le \frac{1}{2}\bar{\delta}(\bar{W}^2 - ||W_e||^2) \tag{27}$$

Substituting Equation (27) into Equation (24), we get

$$\begin{aligned} V' &\le -\frac{1}{2}(W_e^T \Phi(Z))^2 + \frac{1}{2}\bar{\varepsilon}^2 - \frac{1}{2}\bar{\delta}||W_e||^2 + \frac{1}{2}\bar{\delta}\bar{W}^2 \\ &\le -\frac{1}{2}\bar{\delta}||W_e^T||^2 + \frac{1}{2}\bar{\varepsilon}^2 + \frac{1}{2}\bar{\delta}\bar{W}^2 \\ &\le -c_1 V + c_2 \end{aligned} \tag{28}$$

where $c_1 = \bar{\delta}\,\Gamma$ and $c_2 = \frac{1}{2}\bar{\varepsilon}^2 + \frac{1}{2}\bar{\delta}\bar{W}^2$. The $c_1 = \bar{\delta}\,\Gamma$ represents the convergence rate coefficient of the Lyapunov function, whose value is jointly determined by the weight correction coefficient $\bar{\delta}$ and the learning rate matrix Γ; The $c_2 = \frac{1}{2}\bar{\varepsilon}^2 + \frac{1}{2}\bar{\delta}\bar{W}^2$ represents a composite quantity representing the upper bound of the network approximation error, where $\bar{\varepsilon}$ and $\bar{W}$ correspond to the minimum approximation error of the neural network and the weight boundary, respectively. ■

Combining Equations (25)–(28), we have

$$V \le V(0) - \frac{c_2}{c_1}e^{-c_1 t} + \frac{c_2}{c_1} \le V(0) + \frac{c_2}{c_1} \tag{29}$$

Define $\chi = 2\left(V(0) + \frac{c_2}{c_1}\right)$, by using Equation (18), it can be seen that the weight estimation error W_e is bounded, and it has

$$||W_{ek}|| \le \sqrt{\chi\Gamma} \tag{30}$$

4. PSO with Q-reinforcement learning

In the MPC algorithm, the values of the prediction time domain n_p and the control time domain n_c will directly affect the performance of the control algorithm. It is difficult to adjust the n_p and n_c manually to simultaneously satisfy the disturbance rejection performance and computational efficiency. The state of reinforcement learning is set to the current velocity and position of the particle, reflecting the change in the global optimal solution. The action is set to the velocity update rule. The reward is set as the difference between the new and old global optimal solution. This strategy effectively improves the search performance of PSO. By integrating the essential properties of Q-reinforcement learning and the PSO algorithm, an intelligent optimisation algorithm with strong robustness and global convergence, suitable for the MPC controller, is designed and summarised as follows.

4.1. Initialise the population

The number of particles in the PSO is set to M. The maximum iteration number is set to N. The position of the particle swarm is $X(0) = [x_1(0), \ldots, x_M(0)]$ where $x_i(0) = [x_{i,1}(0), \ldots, x_{i,D}(0)]$ $(1 \le i \le M)$. The velocity is $V(0) = [v_1(0), \ldots, v_M(0)]$ where $v_i(0) = [v_{i,1}(0), \ldots, v_{i,D}(0)]$, and boundaries of the position and velocity of the particle are set as $[X_{min}, X_{max}]$ and $[V_{min}, V_{max}]$, respectively. The dimensions are defined as D. The current position $X(0)$ is taken as the individual optimal value $X(0) = P(0)$. The particle position G_{best} with the smallest fitness function value is recorded as the current global optimal position x_{best}.

To ensure the controller exhibits superior tracking performance, the CATE is used as the fitness function

$$\mathrm{func}(x_i(k)) = \sum_{j=1}^{N}(e_1(j) + e_2(j) + \cdots + e_D(j)) \tag{31}$$

where $e_h(j) = abs(y_{dh}(j) - y_h(j))$ $(1 \le h \le D)$; $r_h(j)$ is the expected input of dimension h; $y_{dh}(j)$ is the system output of dimension h.

4.2. Initialise the Q-table

Four states $PS_i(1 \le i \le 4)$ and four actions $action_i(1 \le i \le 4)$ are designed. The action of Q-learning is taken as a combination of different parameters. Four actions are taken as shown in Table 1, whereω_1 is the inertia weight; c_1 is the individual acceleration coefficient and c_2 is the global acceleration coefficient.

The four actions in Table 1 can also be written in matrix form as follows:

$$action = \begin{bmatrix} action_1 \\ action_2 \\ action_3 \\ action_4 \end{bmatrix} = \begin{bmatrix} 0.9 & 2.5 & 0.5 \\ 0.8 & 2 & 1 \\ 0.6 & 1 & 2 \\ 0.4 & 0.5 & 2.5 \end{bmatrix} \tag{32}$$

Table 1. Four actions.

	ω_1	c_1	c_2
Global search	0.9	2.5	0.5
Local development	0.8	2	1
Slow convergence	0.6	1	2
Fast convergence	0.4	0.5	2.5

4.3. Compute the current state and determine the action

According to the Euclidean distance $Edist(x_i(k), x_{best})$ between the $x_i(k)$ and x_{best}, the states $PState_i(k)$ are discretised as follows:

$$PState_i(k) = \begin{cases} PS_1, & 0.75 \leq Edist(x_i(k)) \\ PS_2, & 0.5 \leq Edist(x_i(k)) < 0.75 \\ PS_3, & 0.25 \leq Edist(x_i(k)) < 0.5 \\ PS_4, & 0 \leq Edist(x_i(k)) < 0.25 \end{cases} \quad (33)$$

where $PS_i (1 \leq i \leq 4)$ is the state after discretisation.

The action of the particle is determined according to the following greedy strategy.

$$param = \begin{cases} action(randi(1,n),:), & rand() < acf \\ \max Q, & rand() \geq acf \end{cases} \quad (34)$$

where $param = action(aindex\,,:)$, $aindex$ is subscripts; $randi(1, n) \in [1, n]$ is a random integer; $rand() \in [0, 1]$ is a random number; $\max Q$ is the action that can obtain the maximum reward value in the state $PState_i(k)$ according to the current Q-table; acf is the action selection factor, which is used to determine the weight between random actions and selecting actions.

4.4. Update the velocity and position of the particle

The velocity and position of the particle are updated according to the determined action.

$$\begin{aligned} v_i(k+1) &= param(1)v_{i,j}(k) + r_1 \\ &\quad \times param(2)(p_i(k) - x_i(k)) + r_2 \\ &\quad \times param(3)_1(x_{best} - x_i(k)) \end{aligned} \quad (35)$$

$$x_i(k+1) = x_i(k) + v_i(k+1) \quad (36)$$

where $r_1 \in [0, 1]$ and $r_2 \in [0, 1]$ are random numbers.

4.5. Update optimal values and particle state

According to the fitness function $func()$, the local and global optimal values are updated as follows:

$$p_i(k+1) = \begin{cases} p_i(k), & func(x_i(k+1)) \geq func(p_i(k)) \\ x_i(k+1), & func(x_i(k+1)) < func(p_i(k)) \end{cases} \quad (37)$$

$$x_{best} = \begin{cases} x_{best}, & func(x_i(k+1)) \geq G_{best} \\ x_i(k+1), & func(x_i(k+1)) < G_{best} \end{cases} \quad (38)$$

$$G_{best} = \begin{cases} G_{best}, & func(x_i(k+1)) \geq G_{best} \\ func(x_i(k+1)), & func(x_i(k+1)) < G_{best} \end{cases} \quad (39)$$

By using Equation (33) to calculate the state $PState_i(k+1)$ and the current reward value rwd_i

$$rwd_i = func(x_i(k)) - func(x_i(k+1)) \quad (40)$$

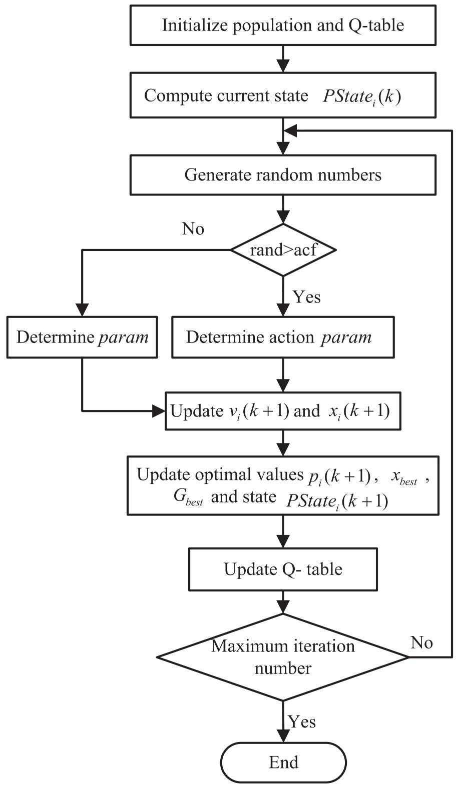

Figure 2. PSO algorithm with Q-reinforcement learning.

4.6. Update the Q-table

$$\begin{aligned} Q(PState_i(k+1), aindex) &= (1-\alpha_p) * Q(PState_i(k), aindex) \\ &\quad + \alpha_p(rwd_i + \gamma_p \max_{aindex} Q(PState_i(k+1),:)) \end{aligned} \quad (41)$$

where α_p is the learning law, which determines the rate at which new information replaces old information; γ_p is the discount factor, which determines how much emphasis the algorithm places on future rewards; $\max_{aindex} Q(PState_i(k+1)\,,:))$ is the largest $PState_i(k+1)$value in the Q-table.

4.7. Repeat steps 3–6 until the maximum iterations number N is satisfied.

The calculation process of the proposed QPSO algor ithm is shown in Figure 2. The diagram of the proposed control algorithm is shown in Figure 3.

5. Stability analysis

The stability of the proposed control algorithm is analysed by constructing the Lyapunov function. At time k, by solving the optimisation problem (9), the optimal control sequence $\{\mathbf{u}^*(k), \ldots, \mathbf{u}^*(k+n_c-1)\}$ is obtained, and its corresponding predictive output is

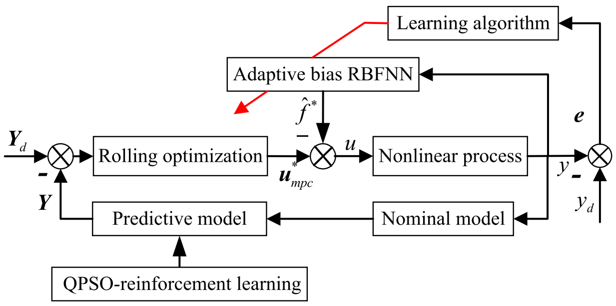

Figure 3. Diagram of the proposed control algorithm.

$\{\mathbf{y}^*(k+1), \ldots, \mathbf{y}^*(k+p)\}$. Therefore, the global criterion function can be expressed as

$$J^*(k) = \sum_{i=1}^{p} \theta_i ||\mathbf{y}^*(k+i|k) - \mathbf{y}_d(k+i)||^2 + \sum_{i=0}^{m-1} \xi_i ||\mathbf{u}^*(k+i|k)||^2 \quad (42)$$

Utilising recursive arithmetic relationships, Equation (42) can also be written as

$$J(k+1) = J^*(k) - \theta_1 ||\mathbf{y}^*(k+1|k) - \mathbf{y}_d(k+1)||^2 - \xi_1 ||\mathbf{u}^*(k|k)||^2 + \theta_1 ||\mathbf{y}^*(k+n_p+1|k) - \mathbf{y}_d(k+n_p+1)||^2 + \xi_1 ||\mathbf{u}^*(k+n_c|k)||^2 \quad (43)$$

Thus

$$J(k+1) \le J^*(k) - \theta_1 ||\mathbf{y}^*(k+1|k) - \mathbf{y}_d(k+1)||^2 - \xi_1 ||\mathbf{u}^*(k|k)||^2 \quad (44)$$

If the optimisation problem has an optimal solution at time $k+1$, it has

$$J^*(k+1) \le J(k+1) \quad (45)$$

Therefore, it has

$$J^*(k+1) \le J(k+1) \le J^*(k) - \theta_1 ||\mathbf{y}^*(k+1|k) - \mathbf{y}_d(k+1)||^2 - \xi_1 ||\mathbf{u}^*(k|k)||^2 \quad (46)$$

According to Equation (46), $\Delta J^*(k+1)$ can be expressed as follows:

$$\Delta J^*(k) = J^*(k+1) - J^*(k) \le -\theta_1 ||\mathbf{y}^*(k+1|k)|| - \xi_1 ||\mathbf{u}^*(k|k)|| \le 0 \quad (47)$$

According to Lyapunov's theory, since $J^*(k) > 0$ is true, $J^*(k)$ is a decreasing function, which means that the closed-loop system is asymptotically stable. This completes the proof.

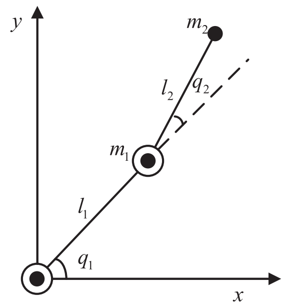

Figure 4. Schematic diagram of the nonlinear manipulator.

6. Experimental verification

6.1. Dynamics modelling nonlinear manipulator

The nonlinear manipulator system studied in this article is composed of two rigid rods connected by joints, and its structure is shown in Figure 4, where l_1 and l_2 are the lengths of the two connecting rods with masses m_1 and m_2, respectively; q_1 and q_2 are the angles of the two joints. Without loss of generality, it is assumed that the masses of the two joints are concentrated at the endpoints. Considering the effects of actual modelling errors and external disturbances, the Lagrange equation is used to establish the dynamics model of the manipulator system with coupling relationships as follows:

$$\mathbf{H}(\mathbf{q})\mathbf{q}' + \mathbf{M}(\mathbf{q}, \mathbf{q}')\mathbf{q}' + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\mathbf{q}') + \boldsymbol{\tau}_\mathbf{d} = \boldsymbol{\tau} \quad (48)$$

where $\mathbf{q} \in R^n$, $\mathbf{q}' \in R^n$ and $\mathbf{q}' \in R^n$ are the joint angles, angular velocities and angular accelerations of the manipulator in space, respectively; $\mathbf{H}(\mathbf{q}) \in R^{n\times n}$ is the inertia matrix of the manipulator; $\mathbf{M}(\mathbf{q}, \mathbf{q}') \in R^n$ is the matrix composed of centrifugal force and Coriolis force; $\mathbf{G}(\mathbf{q}) \in R^n$ is the gravity matrix; $\mathbf{F}(\mathbf{q}') \in R^n$ is the friction matrix; $\boldsymbol{\tau}_\mathbf{d} \in R^n$ is a bounded external disturbance; $\boldsymbol{\tau} \in R^n$ is the control torque.

Without loss of generality, the following three assumptions are given.

Assumption 1: The inertia matrix $\mathbf{H}(\mathbf{q})$ is a symmetric positive definite matrix, which means the existence of positive numbers h_1 andh_2 satisfying the following inequality.

$$h_1||\mathbf{x}||^2 \le \mathbf{x}^T\mathbf{H}(\mathbf{q})\mathbf{x} \le h_2||\mathbf{x}||^2, \quad \forall \mathbf{x} \in R^{n\times n} \quad (49)$$

Assumption 2: $\mathbf{H}'(\mathbf{q}) - 2\mathbf{M}(\mathbf{q}, \mathbf{q}')$ is a skew symmetric matrix and satisfies

$$\mathbf{x}^T[\mathbf{H}'(\mathbf{q}) - 2\mathbf{M}(\mathbf{q}, \mathbf{q}')]\mathbf{x} = 0 \quad (50)$$

The mechanism model of the nonlinear manipulator can be established through analysis, assumption, abstraction and simplification. In practice, it is difficult to accurately identify and obtain the parameters of the dynamic model because of uncertainty disturbances and friction. Therefore, the manipulator system model can be expressed in the form of the following nominal model combined with uncertainties, i.e.

$$\begin{cases} \mathbf{H}(\mathbf{q}) = \mathbf{H_0}(\mathbf{q}) + \Delta\mathbf{H}(\mathbf{q}) \\ \mathbf{M}(\mathbf{q}, \mathbf{q}') = \mathbf{M_0}(\mathbf{q}, \mathbf{q}') + \Delta\mathbf{M}(\mathbf{q}, \mathbf{q}') \\ \mathbf{G}(\mathbf{q}) = \mathbf{G_0}(\mathbf{q}) + \Delta\mathbf{G}(\mathbf{q}) \end{cases} \tag{51}$$

where $\mathbf{H_0}(\mathbf{q}) = \begin{bmatrix} H_1 & H_2 \\ H_3 & H_4 \end{bmatrix}$ is the inertial matrix, $H_1 = (m_1 + m_2)l_1^2 + m_2 l_2^2 + 2m_2 l_1 l_2 \cos q_2 + J_1$, $H_2 = m_2 l_2^2 + m_2 l_1 l_2 \cos q_2, H_3 = m_2 l_2^2 + m_2 l_1 l_2 \cos q_2, H_4 = m_2 l_2^2 + J_2$; J_1 and J_2 are rotational inertia of each joint; $\mathbf{M_0}(\mathbf{q}, \mathbf{q}') = \begin{bmatrix} -m_1 l_1 l_2 q_2' \sin q_2 & -m_2 l_1 l_2 (q_1' + q_2') \sin q_2 \\ 0 & m_2 l_1 l_2 q_1' \sin q_2 \end{bmatrix}$ is the centrifugal and Coriolis force matrix; $\mathbf{G_0}(\mathbf{q}) = \begin{bmatrix} m_1 g l_1 \cos q_1 + m_2 g (l_1 \cos q_1 + l_2 \cos(q_1 + q_2)) \\ m_2 g l_2 \cos(q_1 + q_2) \end{bmatrix}$ is the gravity matrix of the nominal model; g is the gravitational constant; $\Delta\mathbf{H}(\mathbf{q})$, $\Delta\mathbf{M}(\mathbf{q}, \mathbf{q}')$ and $\Delta\mathbf{G}(\mathbf{q})$ are modelling errors of the dynamic model; $\mathbf{q} = [q_1, q_2]^T$, $\mathbf{q}' = [q_1', q_2']^T$.

By substituting Equation (48) into Equation (51), we get

$$\mathbf{H_0}(\mathbf{q})\mathbf{q}' + \mathbf{M_0}(\mathbf{q}, \mathbf{q}')\mathbf{q}' + \mathbf{G_0}(\mathbf{q}) = \boldsymbol{\tau} - f \tag{52}$$

where $f = \boldsymbol{\tau}_\mathbf{d} + \Delta\mathbf{H}(\mathbf{q})\mathbf{q}' + \Delta\mathbf{M}(\mathbf{q}, \mathbf{q}')\mathbf{q}' + \Delta\mathbf{G}(\mathbf{q}) + \mathbf{F}(\mathbf{q}')$ is the lumped error of the dynamic model, and is also named as the total uncertainty.

Define a matrix of state variables $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$ where $x_1 = \mathbf{q}$, $x_2 = \mathbf{q}'$. Define system input variable $\mathbf{u} = \boldsymbol{\tau}$. The nonlinear state space model of the manipulator system can be written as follows:

$$\begin{cases} \mathbf{x}' = \begin{bmatrix} \mathbf{x_2} \\ \mathbf{H_0^{-1}}(\mathbf{x}_1)(\mathbf{u} - \mathbf{M_0}(\mathbf{x}_1, \mathbf{x}_2) - \mathbf{G_0}(\mathbf{x}_1)) \end{bmatrix} \\ \mathbf{y} = C\mathbf{x} \end{cases} \tag{53}$$

The first-order Taylor formula is applied to the model at the sampling point, and the Jacobian matrix of the system model for the state vector is calculated. The linearised system is discretised and the linear model is obtained as follows:

$$\begin{cases} \mathbf{x}(k+1) = A\mathbf{x}(k) + B\mathbf{u}(k) + L_p \\ \mathbf{y}(k) = C\mathbf{x}(k) \end{cases} \tag{54}$$

where $A = \begin{bmatrix} \mathbf{I}_{n\times n} & T\mathbf{I}_{n\times n} \\ \mathbf{O}_{n\times n} & \mathbf{I}_{n\times n} \end{bmatrix}$ is the state matrix;$B = \begin{bmatrix} \frac{T^2}{2}\mathbf{H_0^{-1}}(\mathbf{x}_1(k)) \\ T\mathbf{H_0^{-1}}(\mathbf{x}_1(k)) \end{bmatrix}$ is the input matrix; $L_p = \begin{bmatrix} \frac{T^2}{2}\mathbf{H_0^{-1}}(\mathbf{x}_1(k))(-\mathbf{M_0}(\mathbf{x}_1(k), \mathbf{x}_2(k)) - \mathbf{G_0}(\mathbf{x}_1(k))) \\ T\mathbf{H_0^{-1}}(\mathbf{x}_1(k))(-\mathbf{M_0}(\mathbf{x}_1(k), \mathbf{x}_2(k)) - \mathbf{G_0}(\mathbf{x}_1(k))) \end{bmatrix}$ is the constant term; $T$is the sampling period.

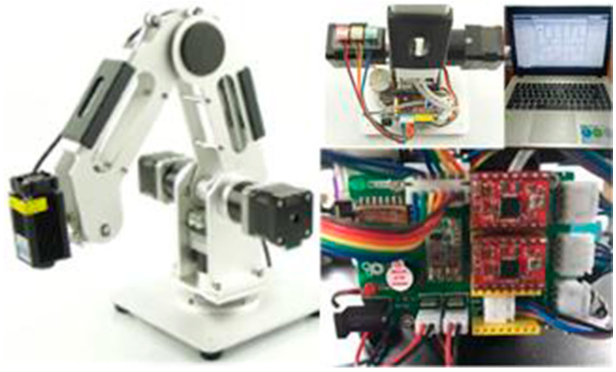

Figure 5. Nonlinear manipulator system test platform.

Table 2. Parameters of the nonlinear system.

Variable	Parameter	Variable	Parameter
m_1	0.3 Kg	g	9.8(m/s^2)
m_2	0.5 Kg	Maximum load	0.5 Kg
l_1	0.2 m	Positioning accuracy	0.3 mm
l_2	0.3 m	Power supply voltage	220 V
J_1	0.05(Kg·m)	Power frequency	50 Hz
J_2	0.1(Kg·m)	Maximum power	60 W

6.2. Control test of nonlinear manipulator

To verify the effectiveness and control performance of the proposed control algorithm, the manipulator system shown in Figure 5 is used for testing. According to sensor measurements and equipment manual, the system parameters that can be determined are shown in Table 2. The initial joint angle of the system is set to $\mathbf{q} = \begin{bmatrix} 2 & 2 \end{bmatrix}$ rad. The initial joint angular velocity is set to $\mathbf{q}' = \begin{bmatrix} 0 & 0 \end{bmatrix}$ rad/s. The sampling interval is taken as $T = 0.01$ s. The designed reference output vector is $y_d = [2 + 2\sin(0.01\pi t), 2 + 2\cos(0.01\pi t)]^T$. The model of the manipulator is obtained through mechanism analysis, which is used as a reference for designing a nonlinear model predictive controller.

The parameters of the NMPC are taken as $\Theta = diag\{100, \ldots, 100\}$ and $\Xi = diag\{0.01, \ldots, 0.01\}$. Since the QPSO optimisation algorithm exists inherent randomness. Ten independent experiments have been conducted to check the robustness of the optimisation algorithm. In these experiments, 10 different random seeds are applied to independently initialise the QPSO parameters. The test results for n_p and n_c are shown in Figures 6 and 7. We can see that the 10 independent experiments converged to the same values. The prediction time domain and the control time domain are converged to $n_p = 9$ and $n_c = 5$, respectively. This result clearly demonstrates that the QPSO algorithm exhibits excellent robustness and stable convergence characteristics when addressing the specific controller parameter optimisation problem in this study. Different random initialisations do not lead to significant deviations or divergence in the optimisation results. For the neural network compensator, the estimation value $\hat{W}$ of the weight in the network output layer is initialised using

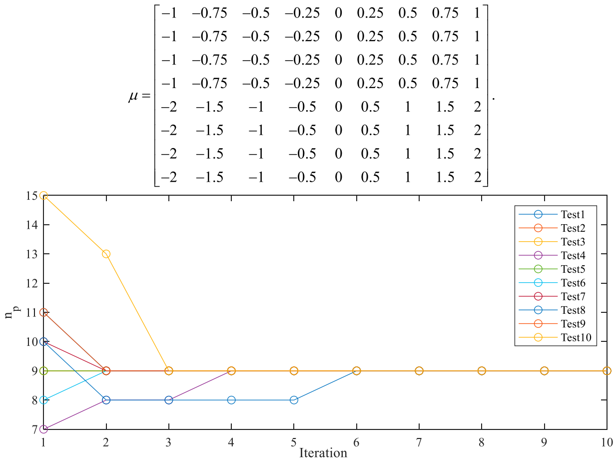

Figure 6. Optimisation results for prediction time domain under different random seeds.

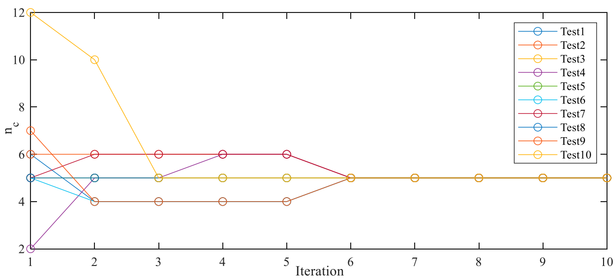

Figure 7. Optimisation results for control time domain under different random seeds.

random values of the standard normal distribution. The parameters are taken as $b_l = 0.1$,$b_g = 1$,$\delta = 0.01$,$\bar{W} = 10$, $\sigma = 10$ and $\Gamma = diag\{50, \ldots, 50\}$. The numbers of input layers, hidden layers and output layers are taken as 8, 10 and 2, respectively. The central position vector of the Gaussian basis function of the hidden layer neuron is taken as follows:

$$\mu = \begin{bmatrix} -1 & -0.75 & -0.5 & -0.25 & 0 & 0.25 & 0.5 & 0.75 & 1 \\ -1 & -0.75 & -0.5 & -0.25 & 0 & 0.25 & 0.5 & 0.75 & 1 \\ -1 & -0.75 & -0.5 & -0.25 & 0 & 0.25 & 0.5 & 0.75 & 1 \\ -1 & -0.75 & -0.5 & -0.25 & 0 & 0.25 & 0.5 & 0.75 & 1 \\ -2 & -1.5 & -1 & -0.5 & 0 & 0.5 & 1 & 1.5 & 2 \\ -2 & -1.5 & -1 & -0.5 & 0 & 0.5 & 1 & 1.5 & 2 \\ -2 & -1.5 & -1 & -0.5 & 0 & 0.5 & 1 & 1.5 & 2 \\ -2 & -1.5 & -1 & -0.5 & 0 & 0.5 & 1 & 1.5 & 2 \end{bmatrix}.$$

To verify the superiority of the proposed algorithm, the SMC algorithm based on RBF compensation in (Zhu et al., 2024), the PID control algorithm in Zhaoet al. (2025) and the NMPC algorithm in Caiaffa et al. (2025) without compensation are also tested for comparison. The controller should be set according to the requirements of the system while satisfying the criteria of rapidity, stability and accuracy. The PID control algorithm parameters are obtained as $k_p = 50.9748$, $k_i = 0.204$ and $k_d = 5.1274$ by using the optimisation algorithm. In the SMC control algorithm, the proportional gain adjustment parameter is set to $\rho_1 = \rho_2 = 100$. The parameters of the sliding mode surface are taken as $\alpha = diag(-1.315, -1.308)$, $\beta = diag(-0.509, -0.504)$ and $\eta = diag(-0.044, -0.045)$. The mean square error (MSE) $\mathrm{MSE} = \frac{1}{N}\sum_{k=1}^{k=N} e^2(k)$ where $e(k) = y(k) - y_d(k)$, average absolute error (MAE) $\mathrm{MAE} = \frac{1}{N}\sum_{k=1}^{k=N} |e(k)|$ and average coupling error (MCE) $\mathrm{MCE} = \frac{1}{N}\sum_{k=1}^{N}\sqrt{\begin{pmatrix} y_1(k)-y_{d1}(k) \\ y_2(k)-y_{d2}(k) \end{pmatrix}^T}$ are defined as performance indicators to analyse the effect of different control algorithms.

Tracking results of joint angles by different control algorithms are shown in Figure 8. Tracking errors of different angles are shown in Figure 9. The tracking error indexes of different algorithms are shown in Table 3. The SMC, PID and NMPC in Figure 8 have large dynamic tracking errors and volatility. The control error of PID algorithm is the largest. The error tracking results in Figure 9 show that the proposed algorithm has the smallest deviation in the two connecting rods.

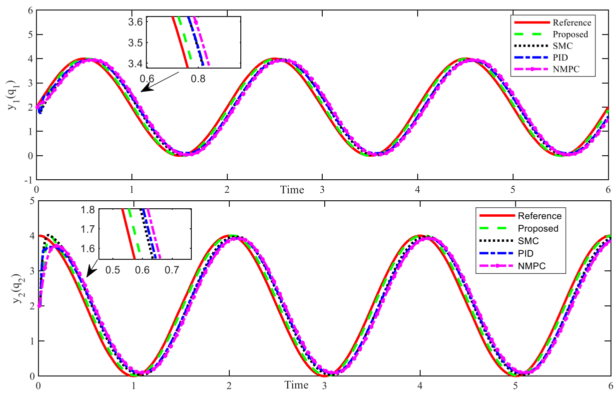

Figure 8. Tracking results of joint angles by different control algorithms.

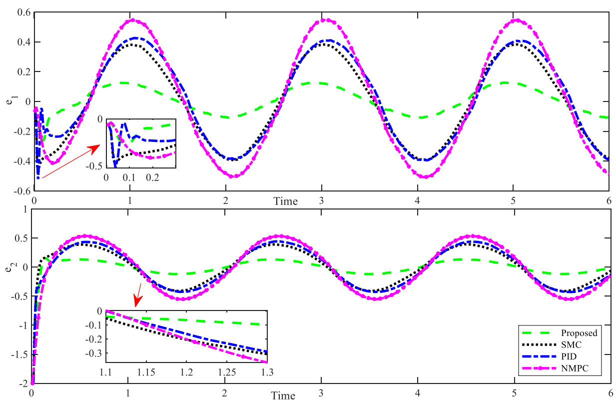

Figure 9. Angle tracking errors.

Table 3. Control error values of different algorithms.

Algorithm	Joint	MSE	MAE	MCE
Proposed	Joint 1	4.50	0.08	0.14
	Joint 2	18.01	0.10	
SMC	Joint 1	44.04	0.24	0.40
	Joint 2	58.26	0.27	
PID	Joint 1	47.81	0.26	0.42
	Joint 2	68.14	0.29	
NMPC	Joint 1	78.62	0.32	0.54
	Joint 2	106.01	0.37	

It can be observed that the average coupling error of the proposed control algorithm is the smallest, which indicates that the end of the manipulator can track the target trajectory with the smallest error. The PID control algorithm cannot use the model prediction value to improve the controller performance. Since PID is designed based on the error feedback of linear systems, it is difficult for PID to accurately capture all the dynamic characteristics of nonlinear systems. The PID control output still has some static error and overshoot after the system reaches a steady state. PID controllers are difficult to adjust automatically to adapt the changes in system parameters or environmental conditions. Since PID control is suitable for single-input single-output systems, two PID controllers are used in parallel to control each joint separately in this test. However, this approach does not take into account the strong coupling property of the system, which leads to a large average coupling error. In practical applications, SMC needs to weaken chattering by improving the sliding mode reaching law, which increases the difficulty and complexity of implementation. The control signal used by the SMC controller needs high-frequency switching, which makes the SMC algorithm difficult to track the dynamic setting output stably. Although the coupling error between outputs has been considered in the SMC with RBF, Figures 8 and 9 show that there is a delay in the control process, the control accuracy of each axis is low, and the average coupling error is relatively large. The performance of the basic NMPC control algorithm heavily depends on the accuracy of the model. Any inaccuracy of the model parameters may affect the control accuracy. Due to the high computational complexity, the NMPC algorithm may suffer from delay in the control process. Due to the modelling errors and uncertainty disturbances in the established nominal model, the input calculated by the NMPC technology is not optimal at each moment in the rolling optimisation process, which leads to the largest tracking error by the NMPC without fault tolerance compensation.

It should be noted that compared with fuzzy logic and other compensation technologies, the compensation technology based on adaptive RBFNN has unique advantages. For the fuzzy logic compensator. The fuzzy sets and rules are initialised with the best engineering knowledge available (Al-Kaf et al., 2023). However, accurate a priori rules for the unknown faults and disturbances are difficult to obtain. Thus, the resulting rule base inevitably contains gaps and inaccuracies. Under these conditions, the fuzzy compensator is unable to attenuate the disturbances satisfactorily compared with the proposed adaptive-bias RBFNN scheme.

To test the speed of the proposed algorithm, a square wave signal tracking test is also performed. The partial parameters of NMPC are taken as $\Theta = diag\{500, \ldots, 500\}$ and $\Xi = diag\{0.01, \ldots, 0.01\}$. The designed reference output vector is $y_d = \begin{bmatrix} 5 * (\mathrm{sign}(t-200) - \mathrm{sign}(t-400)) + 5 \\ 10 * (\mathrm{sign}(t-200) - \mathrm{sign}(t-400)) + 5 \end{bmatrix}^T$. The parameters of the RBFNN compensator are taken as $\Gamma = diag\{10, \ldots, 10\}$, $b_l = 0.05$, $b_g = 1$, $\delta = 0.01$ and $\bar{W} = 10$. The remaining parameters are kept constant. The tracking results and errors of joint angles are shown in Figure 10. It can be seen from Figure 10 that

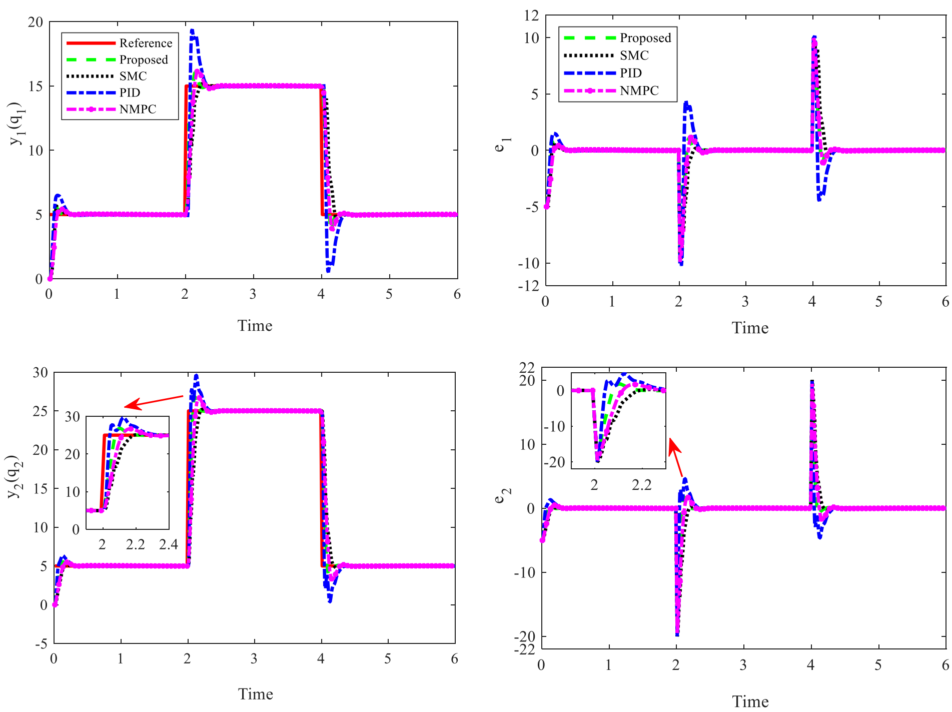

Figure 10. Tracking results and errors of joint square wave angles by different algorithms.

the compared algorithm fails to make good use of the output signal predicted by the model or compensate for the uncertainty deviation by intelligent networks. Therefore, these algorithms have great defects in the rapidity, stability and precision of control tracking. The control results of PID algorithm are the worst. When the setting angle changes, the proposed controller can respond quickly and make the joint angle converge to the setting angle quickly, showing high stability and low steady-state error. Other control algorithms either converge slowly or have steady-state errors. The test results of two different kinds of tracking signals show that the proposed control has better comprehensive control performance.

7. Conclusion

Since it is difficult to accurately model the dynamic model of the nonlinear process. Besides, the nominal models established based on local approximation and simplification techniques may suffer from large modelling deviation. The MPC controller has been designed to achieve the stability control of the nonlinear process. To reduce the influence of modelling deviations, uncertainty disturbances and faults on control precision, an adaptive-bias RBFNN is designed to compensate lumped uncertainties. A bias term is added to the RBF hidden layer to prevent approximation failure when the neural network input deviates from its approximation domain. The QPSO algorithm is constructed to obtain the optimal parameters of the NMPC controller. In the QPSO algorithm, the CATE is applied as the fitness function to ensure that the controller has better tracking accuracy. By constructing the Lyapunov function, it is proven that the proposed control algorithm can converge in finite time. Finally, the control performance of the proposed algorithm is verified by using comparative experiments. The test results show that compared to PID, SMC and NMPC without RBF, the proposed control algorithm has good tracking performance. The test results indicate that the MCE of the control algorithm proposed is smaller. The proposed control algorithm facilitates controller implementation in engineering practice and can be extended to other processes. In future research, the model predictive controllers can be developed to handle constraint saturation. Besides, the artificial intelligence and MPC can be combined to optimise the control performance.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by the Postdoctoral Fellowship Program of CPSF under Grant Numbers GZC20240231 and 2025M773596, Fundamental Research Funds for the Central Universities under Grant Number N25LPY019 and National Natural Science Foundation of China under Grant Number U24A2098.

Notes on contributors

Jianguo Dong was born in Linquan, China. He is currently a student in the School of Automation, Southeast University, Nanjing, China. His main research interests include artificial intelligence and industrial process controller design. His email address is djgseu@163.com

Tianyu Yu was born in Nanchang, China. He is currently a student in the School of Information and Control Engineering, China University of Mining and Technology, Xuzhou, China. His main research interests are intelligent modelling and control of industrial processes. His email address is 07222981@cumt.edu.cn

Shiyu Yang was born in Yaan, China. She is currently a student in the School of Mechanical and Electrical Engineering, China University of Mining and Technology, Xuzhou, China. Her main research interests include artificial intelligence and industrial control. His email address is 03224121@cumt.edu.cn

Ruixian Su was born in Xuzhou, China. She is currently a student in the School of Information and Control Engineering, China University of Mining and Technology, Xuzhou, China. Her main research interests include artificial intelligence and process control. His email address is 03210889@cumt.edu.cn

Shijian Dong (Member, IEEE) was born in Fuyang, China. He received the B.Sc. degree and the M.Sc. degrees from the Northeastern University, Shenyang, China, in 2011 and 2013, respectively. He received the Ph.D. degree in control theory and control engineering from the Dalian University of Technology, Dalian, China, in 2018. He is currently a lecturer in the School of Metallurgy, Northeastern University, Shenyang, China. His main research interests include system modelling, optimisation and predictive control. His email address is dsjggy@126.com

Data availability statement

The data that support the findings of this study are available from the author Shijian Dong (dsjggy@126.com), upon reasonable request.

ORCID

Shijian Dong http://orcid.org/0000-0002-1752-5810

References

Al-Kaf, H. A. G., Halabi, L. M., & Lee, K. B. (2023). A generalized integrated MPC-fuzzy-neural network approach for multilevel inverter fed PMSMs. *IEEE Transactions on Industrial Informatics*, *20*(2), 2751–2761. https://doi.org/10.1109/TII.2023.3297662

Amin, A. A., & Hasan, K. M. (2019). A review of fault tolerant control systems: Advancements and applications. *Measurement*, *143*, 58–68. https://doi.org/10.1016/j.measurement.2019.04.083

Arıcı, M., & Kara, T. (2020). Robust adaptive fault tolerant control for a process with actuator faults. *Journal of Process Control*, *92*, 169–184. https://doi.org/10.1016/j.jprocont.2020.05.005

Bamimore, A. (2024). Application of gap metric for model selection in linear parameter varying model-based predictive control. *Journal of Control and Decision*, *11*(4), 707–729. https://doi.org/10.1080/23307706.2023.2288639

Bangi, M. S. F., & Kwon, J. S. I. (2023). Deep hybrid model-based predictive control with guarantees on domain of applicability. *AIChE Journal*, *69*(5), e18012. https://doi.org/10.1002/aic.18012

Bingül, Z., & Karahan, O. (2011). A fuzzy logic controller tuned with PSO for 2 DOF robot trajectory control. *Expert Systems with Applications*, *38*(1), 1017–1031. https://doi.org/10.1016/j.eswa.2010.07.131

Caiaffa, L., Maran, F., Furlan, M., Peron, S., Beghi, A., & Bruschetta, M. (2025). Anti-wheelie systems for high-performance motorcycles: A nonlinear model predictive control approach. *Control Engineering Practice*, *157*, 106224. https://doi.org/10.1016/j.conengprac.2024.106224

Csekő, L. H., Kvasnica, M., & Lantos, B. (2015). Explicit MPC-based RBF neural network controller design with discrete-time actual Kalman filter for semiactive suspension. *IEEE Transactions on Control Systems Technology*, *23*(5), 1736–1753. https://doi.org/10.1109/TCST.2014.2382571

Ding, S. X., & Li, L. (2021). Control performance monitoring and degradation recovery in automatic control systems: A review, some new results, and future perspectives. *Control Engineering Practice*, *111*, 104790. https://doi.org/10.1016/j.conengprac.2021.104790

Diwan, S. P., & Deshpande, S. S. (2023). Fast nonlinear model predictive controller using parallel PSO based on divide and conquer approach. *International Journal of Control*, *96*(9), 2230–2239. https://doi.org/10.1080/00207179.2022.2087739

Fang, X., Fan, H., Liu, L., & Wang, B. (2025). Decentralized adaptive fault-tolerant control of interconnected systems with sensor faults. *Journal of Control and Decision*, *12*(2), 289–305. https://doi.org/10.1080/23307706.2023.2227184

Fang, X., Ruan, Z., Zhao, S., & Liu, F. (2023). Conditional disturbance-compensation control for an overactuated manned submersible vehicle. *IEEE Transactions on Industrial Informatics*, *20*(3), 4828–4838. https://doi.org/10.1109/TII.2023.3329682

Gan, W., Zhu, D., & Ji, D. (2018). QPSO-model predictive control-based approach to dynamic trajectory tracking control for unmanned underwater vehicles. *Ocean Engineering*, *158*, 208–220. https://doi.org/10.1016/j.oceaneng.2018.03.078

Guo, G., & Zhang, C. L. (2024). Adaptive fault-tolerant control with global prescribed performance of strict-feedback systems. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *54*(8), 4832–4840. https://doi.org/10.1109/TSMC.2024.3388729

Hill, E., Gadsden, S. A., & Biglarbegian, M. (2021). Explicit nonlinear MPC for fault tolerance using interacting multiple models. *IEEE Transactions on Aerospace and Electronic Systems*, *57*(5), 2784–2794. https://doi.org/10.1109/TAES.2021.3065089

Hsieh, Y. Z., & Su, M. C. (2016). A Q-learning-based swarm optimization algorithm for economic dispatch problem. *Neural Computing and Applications*, *27*(8), 2333–2350. https://doi.org/10.1007/s00521-015-2070-1

Júnior, G. A. N., Martins, M. A., & Kalid, R. (2014). A PSO-based optimal tuning strategy for constrained multivariable predictive controllers with model uncertainty. *ISA Transactions*, *53*(2), 560–567. https://doi.org/10.1016/j.isatra.2013.12.019

Köhler, J., Soloperto, R., Müller, M. A., & Allgöwer, F. (2021). A computationally efficient robust model predictive control framework for uncertain nonlinear systems. *IEEE Transactions on Automatic Control*, *66*(2), 794–801. https://doi.org/10.1109/TAC.2020.2982585

Krog, H. A., & Jäschke, J. (2024). A simple and fast robust nonlinear model predictive control heuristic using n-steps-ahead uncertainty predictions for back-off calculations. *Journal of Process Control*, *141*, 103270. https://doi.org/10.1016/j.jprocont.2024.103270

Li, X., Luo, Q., Wang, L., Zhang, R., & Gao, F. (2022). Off-policy reinforcement learning-based novel model-free minmax fault-tolerant tracking control for industrial processes. *Journal of Process Control*, *115*, 145–156. https://doi.org/10.1016/j.jprocont.2022.05.006

Li, Y. L., Shao, W., You, L., & Wang, B. Z. (2013). An improved PSO algorithm and Its application to UWB antenna design. *IEEE Antennas and Wireless Propagation Letters*, *12*, 1236–1239. https://doi.org/10.1109/LAWP.2013.2283375

Lin, Y., McPhee, J., & Azad, N. L. (2020). Comparison of deep reinforcement learning and model predictive control for adaptive cruise control. *IEEE Transactions on Intelligent Vehicles*, *6*(2), 221–231. https://doi.org/10.1109/TIV.2020.3012947

Liu, L., Yuan, P., & Mao, Z. (2024). Precision positioning control of wind tunnel centrosome. *IET Control Theory & Applications*, *18*(5), 659–678. https://doi.org/10.1049/cth2.12599

Papadimitrakis, M., & Alexandridis, A. (2022). Active vehicle suspension control using road preview model predictive control and radial basis function networks. *Applied Soft Computing*, *120*, 108646. https://doi.org/10.1016/j.asoc.2022.108646

Rezvani, N., & Keymasi-Khalaji, A. (2024). Adaptive RBFNN-based predictive control for the nanopositioning of an electrostatic MEMS actuator. *IET Control Theory & Applications*, *18*(5), 551–565. https://doi.org/10.1049/cth2.12586

Shabbir, W., Aijun, L., & Yuwei, C. (2023). Neural network-based sensor fault estimation and active fault-tolerant control for uncertain nonlinear systems. *Journal of the Franklin Institute*, *360*(4), 2678–2701. https://doi.org/10.1016/j.jfranklin.2022.12.044

Souza, D., Odloak, G., Zanin, D., & C, A. (2010). Real time optimization (RTO) with model predictive control (MPC). *Computers & Chemical Engineering*, *34*(12), 1999–2006. https://doi.org/10.1016/j.compchemeng.2010.07.001

Sun, Y., Xu, J., Lin, G., Ji, W., & Wang, L. (2022). RBF neural network-based supervisor control for maglev vehicles on an elastic track with network time delay. *IEEE Transactions on Industrial Informatics*, *18*(1), 509–519. https://doi.org/10.1109/TII.2020.3032235

Tian, D., & Shi, Z. (2018). MPSO: Modified particle swarm optimization and its applications. *Swarm and Evolutionary Computation*, *41*, 49–68. https://doi.org/10.1016/j.swevo.2018.01.011

Tijjani, S., Ab Wahab, M. N., & Noor, M. H. M. (2024). An enhanced particle swarm optimization with position update for optimal feature selection. *Expert Systems with Applications*, *247*, 123337. https://doi.org/10.1016/j.eswa.2024.123337

Wang, J. S., & Yang, G. H. (2016). Data-driven output-feedback fault-tolerant control for unknown dynamic systems with faults changing system dynamics. *Journal of Process Control*, *43*, 10–23. https://doi.org/10.1016/j.jprocont.2016.04.004

Wu, W., Niu, B., Xu, N., Zhao, X., & Liu, Y. (2024). Performance recovery-based adaptive saturated fault-tolerant control of uncertain nonlinear systems with actuator switching and periodic disturbances. *European Journal of Control*, *77*, 100969. https://doi.org/10.1016/j.ejcon.2024.100969

Wu, Z., Rincon, D., & Christofides, P. D. (2020). Process structure-based recurrent neural network modeling for model predictive control of nonlinear processes. *Journal of Process Control*, *89*, 74–84. https://doi.org/10.1016/j.jprocont.2020.03.013

Yan, Y., Zhang, H., Zhao, W., & Li, M. (2024). Fault reconstruction algorithm for fractional-order nonlinear switching systems based on optimal fault-tolerant control. *IEEE Transactions on Cybernetics*, *54*(10), 6158–6168. https://doi.org/10.1109/TCYB.2024.3426622

Yang, C., Chen, B., Bu, L., Zhou, L., & Ma, L. (2023). Low-order dynamical model and distributed coordinated model predictive control for multi-stage belt conveyor systems. *Journal of Process Control*, *124*, 83–91. https://doi.org/10.1016/j.jprocont.2023.02.010

Yang, J. Z., Li, Y. X., & Tong, S. (2022). Adaptive asymptotic fault-tolerant tracking of uncertain nonlinear systems with unknown control directions. *Journal of Control and Decision*, *9*(3), 301–310. https://doi.org/10.1080/23307706.2021.1970035

Yang, L., Yue, M., Liu, Y., & Guo, L. (2020). RBFNN based terminal sliding mode adaptive control for electric ground vehicles after tire blowout on expressway. *Applied Soft Computing*, *92*, 106304. https://doi.org/10.1016/j.asoc.2020.106304

Zhang, H., Li, S., & Zheng, Y. (2020). Q-learning-based model predictive control for nonlinear continuous-time systems. *Industrial & Engineering Chemistry Research*, *59*(40), 17987–17999. https://doi.org/10.1021/acs.iecr.0c02321

Zhang, H., Wei, X., Han, J., Hu, X., & Han, J. (2025). Finite-time control for systems subject to multiple disturbances. *Journal of Control and Decision*, *12*(1), 142–149. https://doi.org/10.1080/23307706.2023.2233749

Zhang, H., Yue, X., Sun, J., & Guo, X. (2024). Prescribed finite-time fuzzy consensus control for multiagent systems with aperiodic updates. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *55*(2), 1362–1373. https://doi.org/10.1109/TSMC.2024.3497961

Zhang, J. X., & Yang, G. H. (2020). Fault-tolerant output-constrained control of unknown Euler–Lagrange systems with prescribed tracking accuracy. *Automatica*, *111*, 108606. https://doi.org/10.1016/j.automatica.2019.108606

Zhao, C., Wang, D., & Xue, W. (2025). Beyond linear limits: Design of robust nonlinear PID control. *Automatica*, *173*, 112075. https://doi.org/10.1016/j.automatica.2024.112075

Zhu, M., Huang, C., Song, S., Xu, S., & Gong, D. (2024). Vision-admittance-based adaptive RBFNN control with a SMC robust compensator for collaborative parallel robots. *Journal of the Franklin Institute*, *361*(4), 106538. https://doi.org/10.1016/j.jfranklin.2023.11.048